

CS240: Lab 9

TAs: Darshan, Shubham, Suvasish

March 30, 2026

General Instructions

1. Welcome to the world of Sequence Modeling. In this **ungraded outlab**, you will implement two of the most influential architectures in modern deep learning, along with some of their powerful extensions. We hope that you find these tasks as exciting to solve as we did when designing them !!
2. Please note that, although this outlab is ungraded, it serves as preparation for the **two upcoming Graded Inlabs**: one focused on RNN (scheduled next week) and another on Transformers (scheduled for the week after). We highly encourage that you work through these problems to ensure you are well-prepared for the graded evaluations.
3. The problem set contains four tasks in total. The first two tasks are foundational, where you are asked to implement RNNs and Transformers. We strongly recommend completing these as they will help you prepare for inlabs. The remaining two questions, along with the homeworks, are completely optional. These advanced tasks are meant to give you a glimpse into the style of reasoning that you can expect in the inlabs. However, the inlabs questions will be significantly easier. We encourage you to go over the solutions we will provide before the next in-lab. Additionally, each task is marked with stars (★) to indicate their hardness.

0 Problem Definition: Learning to solve Prefix-Sum

Our primary goal in this outlab is to explore whether sequence-to-sequence models like RNNs and Transformers can learn simple algorithmic tasks like **Prefix-Sum** (also called **Cumulative-Sum**).

What is the Prefix-Sum problem?

It is a deceptively straightforward problem. You are given an input sequence of N numbers $\mathbf{X} = [x_1, x_2, \dots, x_N]$ and your goal is to produce an output sequence $\mathbf{Y} = [y_1, y_2, \dots, y_N]$, such that each element y_i is the sum of all preceding elements up to that point in X . That is, $y_i = \sum_{j=1}^i x_j$. For example:

Example 1:

Input (X): [1, 2, 3, 4, 5]

Output (Y): [1, 3, 6, 10, 15]

Example 2:

Input (X): [0, 0, 0, 1, 1]

Output (Y): [0, 0, 0, 1, 2]

Recall from your data-structures course that the "trivial" solution to this problem involves maintaining a running total as you iterate through the array (there are also more advanced solutions like Parallel-Scan that solves this in $O(\log N)$ time by leveraging GPU's parallelization). However, the question we are interested in is: *Can sequence-to-sequence models like RNNs and Transformers discover this logic internally?*

To make the problem a little easier, we will have the following constraints:

1. $0 \leq x_i \leq 9$
2. $N = 25$

Your mission should you choose to accept it, is to assist the sequence-to-sequence architectures in mastering this problem.

1 Primer on Sequence-to-Sequence models

1.1 Task 1: Implement RNN [★]

In the class, you have studied Recurrent Neural Networks (RNNs) in detail. Here, you will implement the same from scratch using PyTorch's primitives. Specifically, given an input sequence $\mathbf{X} = [x_1, x_2, \dots, x_N]$, where each $x_i \in \mathbb{R}^d$, you will compute the hidden states and the corresponding outputs according to the following recurrence relations:

$$h_t = \sigma(W h_{t-1} + U x_t + b) \quad (1)$$

$$o_t = V h_t + c \quad (2)$$

where $W, U, V \in \mathbb{R}^{d \times d}$, $h_t, x_t, b, c \in \mathbb{R}^d$ are learnable weight parameters (except h_t, x_t) and σ is the activation function (typically $\tanh$ or ReLU). Please note here that, for simplicity, we make the input, hidden and output dimensions same, but you can initialize them differently as well.

To make your life simple, we are asking you to implement only the forward pass of RNN and let PyTorch's autograd handle the gradient computations. However, note that in high-performance or custom implementations, you will see that the gradients are also computed manually using BPTT (Backpropagation Through Time). Doing so will enable fine-grained control over gradient flow, which is useful to handle vanishing and exploding gradients that naturally arise when unrolling really long sequences. For activation function, you can use `tanh()` function.

Homework 1: What happens if you increase the value of "N"? Also, given that you know the problem at hand, can you also come up with a more appropriate loss function than the generic Cross-Entropy or MSE loss?

Homework 2: Can we go a step further and come up with a "tweaked" RNN formulation (either setting weights in a specific way, or computing hidden states differently) that requires minimum number of trainable parameters while remaining mathematically capable of solving the Prefix-Sum problem?

1.2 Task 2: Implement Transformers [★★]

Similar to the previous task, here you will implement the forward pass of a Transformer-based architecture. Before you begin, if you require a refresher on the Transformer mechanics, here is an excellent tutorial you can follow: Illustration of the Transformer Architecture by Jay Alammar. Assuming you have a conceptual grasp of the architecture, given an input sequence $\mathbf{X} = [x_1, x_2, \dots, x_N]$, where each $x_i \in \mathbb{R}^d$, your task is to implement the following sequence of operations:

Positional Encoding: Since Transformers process all tokens in parallel, they lack an inherent sense of order. To address this, you need to inject sequence information into the input sequence before attention is computed. To do so, for every position i and dimension index j , you can compute the positional encoding as:

$$PE(i, 2j) = \sin(i/10000^{2j/d}) \quad (3)$$

$$PE(i, 2j+1) = \cos(i/10000^{2j/d}) \quad (4)$$

where $PE \in \mathbb{R}^{N \times d}$. Finally, the input is updated as $\mathbf{X}_{enc} = \mathbf{X} + PE$.

Multi-head Attention: With the positional information injected, you can now calculate the scaled dot-product attention. For a single attention head, this can be computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (5)$$

where the Query (Q), Key (K), and Value (V) matrices are linear projections of the input: $Q = \mathbf{X}_{enc}W^Q$, $K = \mathbf{X}_{enc}W^K$, and $V = \mathbf{X}_{enc}W^V$ where $W^Q, W^K, W^V \in \mathbb{R}^{d \times d}$.

To make this Multi-headed, you will perform the same calculation for each head $i \in 1, \dots, H$ and concatenate the results. That is:

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (6)$$

where $\text{head}_i = \text{Attention}(\mathbf{X}_{enc}W_i^Q, \mathbf{X}_{enc}W_i^K, \mathbf{X}_{enc}W_i^V)$. Typically, when you have H heads, each head operates at a subspace of d/H dimensions (that is, you define weight matrices used to compute Q , K , and V such that the dimensions all checks out). Also, instead of manually computing attention for each head, you can use Pytorch's broadcasting to simultaneously compute eq. (5) for each head. We will let you figure this out.

Note: For the Prefix-Sum task, you might also want to apply a causal mask M (where $M_{ij} = -\infty$ for $j > i$ and 0 otherwise). This ensures that position i cannot attend to future elements in the sequence. If you want to do this, then the attention calculation becomes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V \quad (7)$$

Residual Connections and Layer Normalization: Finally, to ensure stable training and mitigate the "vanishing gradient" problem, we utilize two critical components: (1) **Residual Connections**: which allow gradients to bypass layers by adding the input of a sub-layer to its output ($x + \text{Sublayer}(x)$) and (2) **Layer Normalization** that normalizes the activations, ensuring the gradients remain constrained. For the sake of this outlab, these components are already provided in the code template.

Homework 1: Similar to RNNs, can you come up with a "tweaked" Transformers formulation that lets you still solve Prefix-Sum problem with minimum possible parameters?

Homework 2: The sinusoidal embeddings you just implemented is the simplest one that was proposed in the original paper. A lot of research has gone into how to make them more efficient. The three questions you should ask yourself is:

1. can we come up with mechanisms to learn positional embeddings? (read about relative positional embeddings [4])
2. How can we handle sequences longer than the sequences seen during training? (read about AliBi embeddings [3])
3. can we come up with positional embeddings that induces some bias into Attention? (read about RoPE embeddings [5])

2 Advanced questions on RNN and Transformers

The questions in this section are designed to test your research acumen and your ability to reason about the mathematical properties of the architectures you just implemented. While such questions will not be directly asked in graded inlabs (as they are quite hard), you can expect questions of similar flavour to appear in the graded inlabs.

2.1 Task 3: Faster RNNs

[★★★★]

A fundamental trade-off between the two models you implemented in Tasks 1 and 2 are, while RNNs are more memory-efficient ($O(N)$), they are extremely slow for long sequences due to their recurrent nature. In contrast, Transformers are quite fast due to the token level parallelism, but suffer from quadratic memory complexity ($O(N^2)$). Interestingly, few researchers found out that if the non-linearity used by RNNs between the time-steps is removed, you can come up with RNNs that operate at the speed of Transformers, while requiring memory similar to RNNs. Your task is to figure out the trick to achieve this and implement the same. More concretely, the updated RNN formulation looks like this:

$$h_t = Wh_{t-1} + Ux_t + b \quad (8)$$

$$o_t = Vh_t + c \quad (9)$$

NOTE: What you just implemented lays the foundation for something called State Space Models (SSMs). These are a new class of sequence-to-sequence models, popularly known as Linear Attention. The most talked about architecture in this class is Mamba [2], developed by the same researchers behind Flash-Attention (a hardware-aware Attention implementation that is 10-20x faster than the vanilla implementation you just did in Task 2).

2.2 Task 4: Attention visualization and Convolution in Attention [★★★]

In this task, you will first write code to visualize Attention. This will allow you to peek into the attention and understand what patterns are learned by your model. This will let you ascertain if the patterns you see align with the algorithmic solution to the Prefix-Sum problem you know.

Once you are done with the visualization, your task is very simple. Can you tweak the Attention formulation so that it behaves exactly like a 1D-Convolution with d channels and kernel size k ? Note that, although this task looks deceptively simple, you have to factor in several things to make this work.

NOTE: Interestingly, there are several research papers that show that a trained attention, more often than not behaves like convolution [1].

References

- [1] Jean-Baptiste Cordonnier, Andreas Loukas, and Martin Jaggi. On the relationship between self-attention and convolutional layers, 2020.
- [2] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024.
- [3] Ofir Press, Noah Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022.
- [4] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In Marilyn Walker, Heng Ji, and Amanda Stent, editors, *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 464–468, New Orleans, Louisiana, June 2018. Association for Computational Linguistics.
- [5] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.